Table des matières

Cours 09 — Rebâtir le projet de bout en bout 1
Objectifs 2
1. Ce que « fini » veut dire 2
2. La carte du projet 3
3. Ce qui se versionne, ce qui se refait, ce qui se garde 4
4. La reconstruction, son ordre et sa garde 5
5. Refaire n'est pas prouver 7
6. L'incident, par les deux bouts 8
7. La passation 8
Points clés à retenir 10
Exercices 10

Cours 09 — Rebâtir le projet de bout en bout

Théorie · Cours 9/9 — Formation Analytics Engineering · avancé

Où vous êtes : `RAW` → `STAGING` → `INTERMEDIATE` → `MARTS` → `PRESENTATION`. Huit cours ont posé chaque couche, chaque étoile et chaque vérification, un morceau à la fois, sur un entrepôt qu'on n'a jamais effacé. Celui-ci n'ajoute rien : il efface ce qu'Analytics Engineering a construit, le refait à partir du dépôt, prouve qu'il est revenu identique, et le passe à ceux qui le liront.

En 30 secondes :

- Un projet dbt qui passe ses tests n'est pas fini. Il est fini quand il se refait de zéro depuis son dépôt, qu'une preuve dit qu'il est revenu identique, et qu'une autre personne peut le reprendre.
- Le code se versionne ; les couches se refont par `dbt build`. L'histoire des paliers est l'exception : elle ne se refait que parce que la `RAW` garde chaque extrait mensuel.
- La reconstruction tient dans un script de neuf commandes, dans un ordre qui ne se discute pas, et une garde l'empêche de tourner sur un entrepôt déjà construit.
- Refaire n'est pas prouver. Trois preuves s'empilent : les lignes, comparées dans les deux sens ; les témoins, qui opposent l'étoile à un autre chemin ; et l'incident de l'usine, retrouvé par ses deux bouts.

© 2026 iAlexMG. Tous droits réservés.

- Une vue ne prouve rien par ses lignes : elle nomme sa base, et celle d'avant lit les tables du refait.
- Un projet se passe par écrit : le `README.md` du dépôt, `LIVRAISON.md`, une documentation sans trou, et une étiquette `v1.0`.

L'idée du cours. Un entrepôt qu'on n'ose pas effacer n'appartient à personne : il appartient à l'état de la machine où il tourne, et au souvenir de celui qui l'a monté. Le capstone fait exprès le jour où tout disparaît, pendant qu'on a encore de quoi comparer.

Le cours se lit en une séance. Le labo en demande deux : la première refait le projet, la seconde le prouve et le livre.

Objectifs

- Dire ce qui distingue un projet dbt fini d'un projet qui passe ses tests.
- Situer chaque morceau du projet, et le cours qui l'a posé.
- Séparer ce qui se versionne, ce qui se refait et ce qui se garde, et dire à quelle condition l'histoire d'un snapshot se refait.
- Écrire une reconstruction dans un ordre juste, et la protéger par une garde.
- Prouver une reconstruction par les lignes, par des témoins indépendants et par une question métier, et dire ce que chaque preuve ne voit pas.
- Retrouver un incident par les deux bouts d'une matrice de bus.
- Passer un projet : le `README.md`, `LIVRAISON.md`, la documentation, l'étiquette.

---

1. Ce que « fini » veut dire

Labo : `Labo_09_AE.md`, étapes 5 et 6.

Depuis le Cours 03, chaque labo a laissé l'entrepôt un peu plus riche, et jamais on ne l'a effacé. Les schémas `staging`, `marts`, `presentation` et `snapshots` portent donc sept cours d'histoire : des lots rejoués, des essais défaits, une télémétrie chargée en quatre passes. Ils sont justes, les tests l'ont montré à chaque cours. Personne ne sait s'ils sont reproductibles, et c'est une autre question.

Un projet fini tient trois promesses, et aucune ne se voit tant qu'on ne l'a pas mise à l'épreuve.

Promesse	**Ce qu'elle veut dire**	**L'épreuve du capstone**
Il se refait	le dépôt et ce que Data Engineering livre suffisent : rien ne dépend d'un geste fait une fois à la main	effacer les quatre schémas du projet, puis tout relancer depuis une copie neuve du dépôt

© 2026 iAlexMG. Tous droits réservés.

Promesse	Ce qu'elle veut dire	L'épreuve du capstone
Il se prouve	ce qui revient égale ce qui était là, et l'étoile répond encore à ses questions	les lignes dans les deux sens, les témoins, l'incident
Il se passe	une autre personne le reprend sans vous : elle sait quoi avoir, quoi lancer et quoi attendre	le `README.md`, `LIVRAISON.md`, la documentation, l'étiquette `v1.0`

La première est la seule qui trouve des défauts. Tant qu'un projet tourne dans le dossier où on l'a écrit, un geste oublié ne se voit pas : un fichier jamais commité, un paquet installé à la main, un schéma créé un soir pour un essai. Refaire depuis une copie du dépôt, c'est rejouer tous ces gestes d'un coup, par écrit, et voir lesquels manquaient. La copie ne voit que ce que Git suit, exactement comme le runner de la CI au Cours 08.

« De zéro » ne veut pas dire « de rien ». La `RAW` et l'argent restent : ils appartiennent à Data Engineering, et le Cours 12 de Data Engineering fait le même exercice sur la couche qu'il possède. Le capstone d'AE ne refait que ce qu'AE produit. Le profil de connexion reste aussi, hors du dépôt, puisqu'il ne s'y est jamais trouvé.

2. La carte du projet

Labo : `Labo_09_AE.md`, étape 10.

Huit cours ont posé huit morceaux. Mis bout à bout, ils forment une seule chaîne : `RAW` → `STAGING`, treize vues → `INTERMEDIATE`, trois modèles éphémères → `MARTS`, sept dimensions et sept faits → `PRESENTATION`, une vue.

Cours	Ce qu'il a posé	Ce qui le prouve
01	la `RAW` reçue, et le dépôt `02_ae`	22 tables comptées contre la livraison de Data Engineering
02	la conception : six processus, les grains, la matrice de bus	un prototype en SQL, avant tout outil
03	dbt, les sources, les vues de `STAGING`, l'état courant	une commande par numéro malgré le lot livré deux fois
04	l'étoile des ventes, la couche du milieu, les clés par hachage	la valeur du fait égale celle de l'ERP
05	les tests, le test unitaire, le snapshot, le type 2, la documentation, la `PRESENTATION`	la réconciliation, toujours à 0,00 $ après l'historisation

© 2026 iAlexMG. Tous droits réservés.

Cours	Ce qu'il a posé	Ce qui le prouve
06	l'après-vente et l'usine : l'accumulation, l'instantané, le fait sans mesure, les dimensions conformes	les refus hors garantie contre la règle ; l'ERP contre le compteur du convoyeur
07	l'incrémental, la télémétrie, les lots de 2026, deux environnements	un chargement par tranches égale un chargement d'un bloc
08	le linteur, le contrat, la fraîcheur, la CI	`PASS=129` sur la structure vide des sources, à chaque demande de tirage

Deux colonnes, et la seconde compte autant que la première. Chaque cours a construit quelque chose et a aussi dit comment on saurait que c'était juste. Le capstone reprend la seconde colonne entière : elle devient la liste de ce qu'une reconstruction doit rendre.

3. Ce qui se versionne, ce qui se refait, ce qui se garde

Labo : `Labo_09_AE.md`, étape 5.

Tout ce qui touche au projet tombe dans une famille, et la frontière entre elles décide de ce qu'on peut effacer.

Ce qui existe	Où	Versionné	Revient par
les modèles, les tests, le snapshot, les macros, le seed, la CI, les outils, le script, les deux documents	le dépôt `02_ae`, 63 fichiers à la fin du cours	oui	`git clone`
le profil de connexion	le dossier `.dbt` de votre répertoire personnel	jamais	à la main, d'après le `README.md`
les paquets	`dbt_packages/`	non	`dbt deps`, aux versions de `package-lock.yml`
la documentation, le manifeste, les journaux	`target/`, `logs/`	non	toute commande dbt, et `dbt docs generate`
les vues, les tables, la présentation	schémas `staging`, `marts`, `presentation`	non	`dbt build`
l'histoire des paliers	schéma `snapshots`	non	les lots rejoués dans l'ordre, si la source les garde

© 2026 iAlexMG. Tous droits réservés.

Ce qui existe	Où	Versionné	Revient par
la `RAW`, l'argent, le journal des chargements	schémas `raw`, `argent`, `pipeline`	non, et pas ici	Data Engineering, ou le kit 02

La première ligne s'écrit, et c'est la seule qui entre au dépôt. Le profil s'écrit aussi, à la main, et reste sur le poste. Les quatre lignes suivantes se dérivent du code et de la `RAW`, et c'est ce que la reconstruction vérifie. La dernière vient d'ailleurs.

Une ligne ne ressemble pas aux autres, et c'est l'idée la plus importante du cours. Une reconstruction refait un état, pas une histoire. Les tables de `marts` se dérivent de la `RAW` telle qu'elle est aujourd'hui ; l'histoire des paliers se dérive de ce qu'elle était chaque mois depuis 2022. Chez Boréal, elle se refait, parce que Data Engineering garde chaque extrait du référentiel des concessionnaires, 52 lots de janvier 2022 à mars 2026, par choix, depuis son Cours 03. Supposez une source qui ne livre que son état courant, ce que font la plupart des systèmes. La reconstruction passe alors au vert, `PASS=128`, avec 156 versions au lieu de 205 ; et 17 733 lignes de vente changent de palier, pour 94 628 792,88 $, sans que le total bouge d'un cent ni qu'un seul test tombe. C'est le type 1 du Cours 04 revenu par la porte de derrière.

La conséquence est une règle d'exploitation. Tout ce qui se dérive de la source se refait, et on peut l'effacer sans crainte. Ce qu'un snapshot a accumulé d'une source qui ne garde rien ne se refait pas : il se sauvegarde, et c'est la seule table du projet qui l'exige.

Snowflake. D'après la documentation de dbt, consultée le 2026-09-23, toutes les tables que dbt crée sur Snowflake sont *transient* par défaut : sans protection de récupération après suppression, et avec au plus un jour de voyage dans le temps. La page ne fait pas d'exception pour le snapshot, qui est une table. Pour une histoire qu'aucune source ne peut refaire, la configuration `transient: false` la rend permanente, et le voyage dans le temps peut alors aller au-delà d'un jour selon l'édition du compte. L'annexe du Labo 03 dit la différence une fois pour toutes.

Mettre de côté avant d'effacer suit la même logique. La première preuve demande l'entrepôt d'avant, et il n'existe plus une fois effacé : on en garde donc une copie de fichier, environ 90 Mo en une seconde. Sans elle, il ne reste que les témoins et l'incident, et deux lignes échangées contre deux autres, au même total, passeraient.

Sur une autre plateforme. Snowflake met de côté sans copier : `create schema marts_avant clone marts;` fait un clone qui ne coûte que ce qui change ensuite, et le Cours 12 de Data Engineering montre le même geste sur une base entière. Databricks fait de même, table par table : `create table marts_avant.fct_ventes shallow clone marts.fct_ventes;` ne copie que les métadonnées et pointe les fichiers de la source, pour les tables gérées de Unity Catalog à partir de Databricks Runtime 13.3 LTS. D'après leur documentation, consultée le 2026-09-23.

4. La reconstruction, son ordre et sa garde

Labo : `Labo_09_AE.md`, étapes 3, 4 et 6.

Refaire le projet demande neuf commandes, et l'ordre de chacune a une raison.

© 2026 iAlexMG. Tous droits réservés.

Commande	Pourquoi là
`dbt deps`	rien ne compile sans `dbt_utils`, pas même la garde
`dbt run-operation exiger_entrepot_neuf`	avant tout ce qui écrit : la garde doit parler pendant qu'il n'y a encore rien à abîmer
quatre `dbt snapshot`, les lots 2022-01, 2024-06, 2024-07 et 2025-12	l'histoire se monte dans l'ordre : la stratégie `check` compare des valeurs à ce qu'elle a enregistré, pas des dates (Cours 05)
`dbt snapshot`, sans lot	l'état courant, le dernier extrait livré, comme chaque nuit en production
`dbt build --exclude snp_concessionnaire`	`dim_concessionnaire` lit le snapshot, qui doit donc exister et être complet ; le reste suit le graphe
`dbt docs generate`	le catalogue lit les objets construits, donc en dernier

Chaque commande porte `--target prod`, sauf la première, qui ne lit pas l'entrepôt. Sans lui, tout atterrirait dans les schémas `dev_*` du Cours 07, sauf le snapshot, dont le schéma est fixé par sa configuration.

Les neuf commandes vivent dans un fichier, `reconstruire.cmd`, et non dans une mémoire. Un jour de panne, personne ne retape sans faute sept commandes dont quatre ne diffèrent que par un mois. Chaque ligne se termine par `|| exit /b 1`, qui arrête le script au premier échec et rend un code non nul : un script qui continuerait après une erreur finirait sur un vert que rien ne justifie. Sur un serveur, ce seraient les mêmes neuf lignes dans un script shell, ou une tâche du graphe de Data Engineering (Cours 06 de Data Engineering) ; la forme change, l'ordre non.

La garde existe parce que deux dégâts échappent à `dbt build`, et que le capstone les a mesurés.

Ce qui arrive sans la garde	Ce qu'on a mesuré
La reconstruction tourne sur un entrepôt qui porte déjà le projet	le snapshot passe de 205 versions à 303, dont 49 qui finissent avant de commencer ; deux tests de `dim_concessionnaire` tombent, et 41 nœuds sont sautés. Ici, les tests le voient. Sur une source qui ne garde pas ses extraits, l'histoire est perdue quand même
Une table dont le modèle a quitté le dépôt reste dans `marts`	elle survit à `dbt build`, qui ne jette jamais ce qu'il ne connaît pas : mesuré, 19 636 lignes intactes, et `dbt ls` ne la nomme pas. Qui la lisait continue de la lire

La garde compte les objets des quatre schémas du projet et refuse s'il y en a un seul. Elle ne fait rien d'autre, et c'est voulu : effacer n'est pas son travail. Un script qui efface lui-même ce qu'il va refaire effacerait un jour le mauvais entrepôt ; un script qui refuse oblige quelqu'un à décider.

© 2026 iAlexMG. Tous droits réservés.

Reste le temps. La reconstruction entière prend 68 secondes, et la répartition surprend : la construction de 129 nœuds dure une dizaine de secondes, les cinq passes du snapshot quarante. Chaque passe dure huit secondes, dont moins d'une pour traiter 156 lignes ; le reste est le démarrage de dbt, qui relit le projet à chaque commande. On compte les commandes, pas les lignes.

5. Refaire n'est pas prouver

Labo : `Labo_09_AE.md`, étapes 7, 8 et 9.

`Reconstruction : au vert` dit que chaque commande a rendu 0, donc que chaque test est passé. Le Cours 05 a montré ce que vaut ce vert : un snapshot daté de l'heure du rejeu faisait annoncer des ventes fausses par palier, tous les tests au vert. Trois preuves s'empilent donc, chacune voyant ce que les autres ne voient pas.

Preuve	**Ce qu'elle compare**	**Ce qu'elle voit**	**Ce qu'elle ne voit pas**
Les lignes	chaque table refaite contre celle d'avant, par `except all` dans les deux sens ; chaque vue par sa définition	la moindre valeur qui diffère, une ligne en double d'un seul côté, une table orpheline	ce qui était déjà faux avant
Les témoins	l'étoile contre un chemin qui ne passe pas par elle : l'ERP, le compteur du convoyeur, une règle recalculée	une valeur fausse avant comme après, sur ce qu'un témoin couvre	ce qu'aucun témoin ne couvre, comme l'attribution d'une vente à un palier
L'incident	l'étoile contre la question pour laquelle elle existe	une clé conforme cassée, une date convertie dans le mauvais fuseau	une réponse juste pour de mauvaises raisons

Les lignes d'abord. `except all` rend les lignes d'un côté qui manquent de l'autre, en comptant les exemplaires : sans `all`, deux lignes identiques d'un côté contre une seule de l'autre passeraient. Les deux sens comptent, ce qui manque au refait et ce qu'il porte en trop. Seize tables reviennent ainsi à 0 écart, les moyennes horaires de l'usine comprises : même requête, mêmes données, même version de DuckDB, et les nombres à virgule flottante reviennent au dernier chiffre. Sur un autre moteur ou une autre version, l'ordre des additions changerait, et la comparaison se ferait à une tolérance près, comme le Cours 07 de Data Engineering l'enseigne.

Une vue, elle, ne se compare pas par ses lignes, et le piège vaut d'être connu. DuckDB range dans chaque vue le nom de la base qu'elle lit, `boreal.marts.fct_ventes`, et les deux fichiers s'appellent `boreal`. Attachée à côté du refait, une vue de l'entrepôt d'avant lit donc les tables du refait. Mesuré sur une copie amputée exprès : 17 741 lignes retirées de l'ancien fait des ventes, que la comparaison des tables voit toutes, pendant que la vue de présentation de l'ancien entrepôt compte toujours 80 618 lignes. Une vue se compare donc par sa définition ; son contenu suit de sa définition et des tables, qui sont comparées. Le même fait a un revers : refaire dans un fichier qui ne s'appelle plus `boreal` change la définition des quatorze vues, qui sortent toutes différentes.

Les témoins ensuite. La première preuve dit que rien n'a bougé ; elle ne peut pas dire que c'était juste. Les cinq tests singuliers du projet, écrits aux Cours 05 à 07, opposent l'étoile à autre chose qu'elle-même : la valeur des ventes à celle des en-têtes de l'ERP, la production au compteur des convoyeurs, les refus hors

© 2026 iAlexMG. Tous droits réservés.

garantie à la règle recalculée, l'ordre des jalons, le compteur d'heures moteur. Ils ont tourné dans la reconstruction, et on les relance seuls pour les lire : `PASS=5`, 465 924 613,14 $ des deux côtés.

Leur limite est dans la troisième colonne. Aucun témoin ne dit à quel palier une vente appartient : c'est ce que la section 3 a mesuré, 94 628 792,88 $ déplacés sans qu'un test bronche. Contre ce défaut-là, seule la première preuve protège, à condition d'avoir mis l'entrepôt d'avant de côté.

6. L'incident, par les deux bouts

Labo : `Labo_09_AE.md`, étape 9.

La troisième preuve est la question pour laquelle l'étoile de Boréal existe : la trace que le compresseur d'air numéro 2, en dérivant en octobre 2025, a laissée chez les clients. Le Cours 07 y a répondu en connaissant la fenêtre. Le capstone la cherche sans la connaître, par chaque bout de la matrice de bus, et les deux chemins doivent tomber au même endroit.

Depuis l'après-vente, un responsable de la garantie range ses réclamations de refroidissement par semaine d'assemblage des motoneiges vendues. Sur vingt et une semaines, d'août à décembre 2025, dix-neuf en portent de zéro à cinq ; les semaines du 20 et du 27 octobre en portent 16 et 18, sur 119 et 118 motoneiges.

Depuis l'usine, un responsable de la maintenance fixe la bande habituelle de la vibration du compresseur 2 à son 95e centile sur les 120 jours où il a tourné, 3,28 millimètres par seconde pour une médiane de 2,47. Six jours en sortent, du 21 au 28 octobre.

Les deux se rejoignent par la date, que les deux étoiles partagent. Le jour où la vibration monte, les motoneiges assemblées ce jour-là reviennent en garantie ; le lendemain du déclenchement, les deux retombent ensemble. Résumé en deux lignes : les 6 jours hors de la bande ont vu assembler 144 motoneiges, dont 31 ont une réclamation de refroidissement, 21,5 pour cent ; les 105 autres jours, 1 823 motoneiges et 16 réclamations, 0,9 pour cent.

Ce qui rend ce croisement possible a été posé bien avant lui. La date d'une heure de l'usine est celle de Boréal, convertie en toutes lettres au Cours 06, sans quoi une heure sur cinq change de jour. Le véhicule a la même clé dans cinq faits, parce que chacun la recalcule par le même hachage. Et l'actif, la ligne d'assemblage comme le compresseur, est une dimension conforme. Rien de cela n'a été écrit pour l'incident : c'est la matrice de bus du Cours 02, qui tient sur un entrepôt refait de zéro.

Le cours dit aussi ce que cela ne prouve pas. Deux courbes qui montent ensemble ne font pas une cause ; ce qui fait pencher, c'est que la cohorte diverge sur le circuit de refroidissement et sur rien d'autre, ce que la télémétrie a montré au Cours 07 depuis un chemin indépendant. L'entrepôt livre une cohorte identifiée par ses numéros de série, et c'est assez pour un rappel ciblé.

7. La passation

Labo : `Labo_09_AE.md`, étapes 2, 10, 11 et 12.

Un projet se passe par écrit, et chaque écrit a son lecteur.

© 2026 iAlexMG. Tous droits réservés.

Ce qui se passe	Son lecteur	Ce qu'il dit
le `README.md` du dépôt	la personne qui reprendra le code	ce qu'est le dépôt, ce qu'il suppose, les commandes dans l'ordre, ce qui doit sortir, ce qui n'y est pas
`LIVRAISON.md`	les formations qui consomment	chaque produit, son grain, ses comptes, ce qu'il faut savoir avant de s'en servir
la documentation et le `manifest.json`	l'analyste qui cherche une table, l'agent qui écrit une requête	chaque modèle, sa description, ses colonnes, ses tests, son lignage
le contrat de `fct_ventes` et l'`exposure`	Business Intelligence	l'interface promise, et qui en dépend
l'étiquette `v1.0`	quiconque cite une version	l'état livré, qui ne bouge plus quand `main` avance

Le `README.md` est celui qu'on oublie, parce que celui qui l'écrit n'en a pas besoin. Il ne raconte pas le code, que les en-têtes des fichiers expliquent : il dit quoi avoir avant de commencer, la première commande et ce qu'elle affiche, puis la suivante, jusqu'à la preuve. Son test est simple : quelqu'un qui ne vous connaît pas refait le projet en le suivant, sans vous appeler.

La documentation, elle, avait un trou que le Cours 08 a mesuré : seize modèles sur trente et un sans description, toutes les vues de `STAGING` et les trois modèles intermédiaires. Compté plus large, le trou était plus grand : le seed et le snapshot, la seule table du projet qui porte une histoire. Un manifeste ne dit que ce que le projet y écrit, et un agent qui le lit ne sait pas mieux. Le capstone le ferme : une description dit le grain et ce que le modèle a déjà fait à la source, en une phrase, pas son SQL.

Le capstone lui-même passe par une branche et une demande de tirage, comme n'importe quel changement depuis le Cours 08. La CI le juge sur sa forme et sur sa compilation, pas sur ses données : ce sont les trois preuves de la section 5 qui ont jugé le fond, et elles ne tournent que sur un entrepôt qui a des données.

Reste à marquer l'état livré. Une étiquette annotée, `v1.0`, nomme un point de l'historique que d'autres peuvent citer. La numérotation suit celle du contrat : un changement qui casse un consommateur, une colonne renommée de `fct_ventes` par exemple, fera la `v2.0`, et l'`exposure` dit à qui l'annoncer.

Recoupements — ce que ce cours partage avec ses voisines.

- Business Intelligence : l'étoile refaite et prouvée est ce que BI restitue, et le contrat de `fct_ventes` est l'interface qu'elle lit. Le kit 03 la rebâtit pour qui arrive sans avoir fait AE.
- Data Science et Machine Learning Engineering : l'étoile de l'usine et la télémétrie nourrissent la maintenance prédictive, et la cohorte de l'incident est un exemple étiqueté.
- AI Engineering : la documentation sans trou et le `manifest.json` sont ce qu'un agent lit pour savoir ce que l'entrepôt contient. Ce que le capstone livre est exactement ce que le kit 03 reconstruit pour les quatre.

© 2026 iAlexMG. Tous droits réservés.

Points clés à retenir

- Un projet est fini quand il se refait de zéro depuis son dépôt, qu'une preuve dit qu'il est revenu identique, et qu'une autre personne peut le reprendre.
- La reconstruction part d'une copie du dépôt, qui ne voit que ce que Git suit.
- Ce qui se dérive de la source se refait ; l'histoire d'un snapshot ne se refait que si la source garde ses extraits. Sinon, elle se sauvegarde.
- Une reconstruction s'écrit dans un script, dans un ordre justifié, et chaque commande arrête le script si elle échoue.
- Une garde refuse l'entrepôt déjà construit : rejouer une histoire par-dessus l'abîme, et une table orpheline survit à `dbt build`.
- Refaire n'est pas prouver : les lignes dans les deux sens, les témoins, la question métier.
- Une vue se compare par sa définition, jamais par ses lignes quand elle nomme sa base.
- Une matrice de bus bien tenue retrouve un incident par n'importe lequel de ses bouts.
- Un projet se passe par écrit : le `README.md`, `LIVRAISON.md`, une documentation sans trou, une étiquette.

Exercices

Les énoncés et leur corrigé sont dans `Exercices_corriges_09_AE.md`. Faites le labo avant : plusieurs questions portent sur le projet que vous aurez refait vous-même.

© 2026 iAlexMG. Tous droits réservés.